

TP Thread : Lecteur vidéo ogg/theora/vorbis (Pthreads, C Threads, C++ Threads)

SEPC : Ensimag 2A - édition 2024

Le TP commence par deux petits exercices préparatoires, pour ne pas avoir à comprendre les threads dans votre langage, les moniteurs ET déboguer votre code avec le TP complet.

L'objectif du TP est la programmation d'un lecteur de vidéo en utilisant les processus légers (threads) de la bibliothèque de threads POSIX, la bibliothèque thread de C, ou bien celle de C++. Nous avons étudié en travaux dirigés quelques problèmes standards de synchronisation. Vous devez implanter une solution combinant plusieurs de ces problèmes et respectant quelques contraintes.

Les synchronisations seront réalisées avec des moniteurs.

1 ¡ Hello World ! ¢ multi-thread, deux mini-exercices (préparant aussi l'examen de TP)

Le but de cette partie est de vous faire coder, dans un mini-exemple, les quelques différences concernant la programmation multi-thread C et la programmation multi-thread en C++.

Il n'y a, volontairement, dans ces mini-exercices, aucune indication précise sur le code, ni sur la façon de les compiler (faire un petit Makefile est une bonne idée). De même, nous ne fournissons pas de tests automatiques. Heureusement ici, `valgrind ...`, `valgrind --tool=helgrind ...` et `valgrind --tool=drd ...`, et l'affichage de votre programme s'exécutant dans un terminal, devrait être suffisant pour en vérifier la correction.

Nous vous conseillons de faire une version complète dans un langage (soit C, soit C++) puis de faire son portage dans l'autre langage pour en comprendre les ressorts et différences.

La doc des Pthreads est disponible dans les pages de man. Une version étendue de la doc pour les deux interfaces de programmation C est disponible dans les pages info.

```

1 $ man pthread_create
2 $ info libc threads # mode texte
3 $ emacs -q --eval '(info "libc threads")' # clickable
4 $ # "emacs -q": mode quick, sans configuration

```

1.1 Exo 1 : code C et puis C++, ou vice-versa

1. Créer un fichier `hello10.c` (ou `hello10.cpp`). Dans ce fichier, vous devez ajouter deux fonctions. Une des fonctions affiche `Hello World!`, tandis que l'autre affiche `Done!`.
2. Depuis le main, lancer 10 threads faisant `hello` et un thread faisant `done`. Ne pas oublier d'attendre la fin des 11 threads dans le main. Compiler et vérifier que tout fonctionne.
3. Créer un moniteur (fonctions + compteur + mutex + variable de condition) manipulant un compteur avec deux fonctions : une pour incrémenter le compteur de 1 et réveiller les threads en attente lorsque le compteur atteint 10, l'autre pour attendre que le compteur soit à 10.
4. utiliser votre moniteur dans les threads pour garantir que le `Done` s'affiche après les 10 `hello`.

1.2 Exo 2 : allocation dynamique d'un type structuré et multi-threading

Reprendre les codes précédents, en définissant un type structuré contenant le compteur, le mutex et la variable de condition. Allouer dynamiquement la structure (`malloc/new`) et donner son pointeur en argument des threads.

Une difficulté est de bien faire l'initialisation en C avant l'utilisation du moniteur.

L'autre, pour C et C++, est de bien faire la libération de la mémoire contenant la structure après la fin de son utilisation.

1.3 BONUS : detach

Uniquement, pour ceux qui ont terminé l'exercice précédent très rapidement en 20 minutes max, reprendre le code pour détacher les threads `hello` et n'attendre dans le main que le thread `done`.

2 Les versions PThread, C ou bien C++

Ce TP existe en deux variantes principales : C ou bien C++. Dans la variante C, il est possible de le faire avec la bibliothèque PThread (la bibliothèque de référence) ou

bien avec les threads du langage C (intégr  depuis C11). Dans la variante C++, vous utiliserez les threads du langage C++.

Les deux variantes ont volontairement un code tr s similaire (basename des fichiers, noms des fonctions, architecture du logiciel). Les indications sont donn es pour la variante C, mais sont donc valables pour les deux.

Le changement principal dans la version C++ est que les fichiers ont un suffixe `.cc` et `.hpp` au lieu de `.c` et `.h`.

3 D codage d'une vid o ogg/theora/vorbis

Dans un fichier vid o ogg/theora/vorbis, ogg est le format de stockage des donn es brutes, theora est le format de codage vid o et vorbis le format de codage audio.

Le fichier ogg contient des *pages*. Ces pages contiennent des *paquets*. Chaque paquet est associ    un flux (*stream*). Un des flux code l'audio. Un autre flux code la vid o.

Pour d coder une vid o, les op rations suivantes sont donc r alis es :

1. lire un bout du fichier et injecter les donn es dans le d codeur ogg jusqu'  pouvoir en extraire une page compl te
2. injecter la page compl te dans le d codeur ogg
3. extraire du d codeur ogg un paquet complet
4. injecter le paquet dans le d codeur du flux (vorbis ou theora)
5. extraire du d codeur les donn es :  chantillons (*samples*) pour l'audio, une image pour la vid o

Chaque flux commence par des ent tes associ s qui sont utilis s pour la d tection et la description du flux. Mais vous n'aurez pas   g rer les aspects algorithmiques du d codage.

4 Le sujet

Pour vous aider, un squelette de code est fourni. Il fait quelques impasses, par exemple lorsque les performances du processeur sont insuffisantes, mais il devrait  tre fonctionnel pour une vid o « classique ».

Vous devez implanter toute la partie concernant la gestion des threads et leurs synchronisations. Il faut, bien s r, ajouter des structures de donn es et les initialiser correctement. Cela inclut les variables mutex, et les variables de conditions, ainsi que tout ce qui vous semblera n cessaire. Il faudra cr er les threads en leur faisant ex cuter les fonctions ad quates. Il faudra aussi g rer correctement la terminaison.

4.1 Compilation

Le squelette fourni inclut un script `cmake` pour construire automatiquement les Makefile utiles.

Vous devez préalablement également créer et remplir un fichier `AUTHORS` , à la racine du projet, avec vos prénoms, noms et logins Typiquement pour l'équipe Alice Matra-Hachette et Robert Kahn, cela donne un fichier `AUTHORS`

```
1 Alice Matra-Hachette matrahaa
2 Bob Kahn khanb
```

La création du Makefile s'effectue en utilisant `cmake` dans un répertoire dans lequel seront aussi créés les fichiers générés. Le répertoire "build" du squelette sert à cet usage. Tout ce qui apparaît dans "build" pourra donc être facilement effacé ou ignoré. Vous avez à créer vous-même le répertoire `C/build/` (ou `C++/build/`).

La première compilation s'effectue donc en tapant :

```
1 cd ensimag-video
2 cd C # ou cd C++
3 # puis créer le répertoire build
4 cd build
5 cmake ..
6 make
7 make test # C only
8 make check # C only
```

et les suivantes, dans le répertoire `build`, avec

```
1 make
2 make test
3 make check
```

4.1.1 Coccinelle (uniquement pour la variante PThreads)

Le test utilise le logiciel *coccinelle* et cherche des lignes de code C + POSIX Threads qui ressemblent à des bugs : une variable `pthread_mutex_t` différente entre le lock et le unlock dans une même fonction par exemple.

Les lignes s'affichant avec un - au début sont les lignes détectées dans le pattern.

Exemple avec le programme faux suivant :

```
1 #include <pthread.h>
2
3 pthread_mutex_t m,m2;
4 pthread_cond_t c;
5
6 void A() {
7     pthread_mutex_lock(&m);
```

```

8   while(0)
9       pthread_cond_wait(&c , &m);
10  pthread_mutex_unlock(&m2);
11 }

```

Le test affichera :

```

1  diff =
2  --- toto.c
3  +++ /tmp/cocci-output-21774-1f97be-toto.c
4  @@ -4,8 +4,5 @@ pthread_mutex_t m,m2;
5     pthread_cond_t c;
6
7  void A() {
8  -     pthread_mutex_lock(&m);
9     while(0)
10 -     pthread_cond_wait(&c , &m);
11 -     pthread_mutex_unlock(&m2);
12 }

```

Le test fourni ne vérifie pas l'exécution du lecteur. Il faudra prendre votre propre video pour cela.

4.2 Rendu des sources

L'archive des sources que vous devez rendre dans Teide est généré par le makefile créé par cmake :

```

1  cd ensimag-video
2  cd build
3  make package_source

```

Il produit dans le répertoire build, un fichier ayant pour nom (à vos login près) `Ensithreadsvideo-1.0.login1-login2-login3-Source.tar.gz`.

C'est ce fichier tar qu'il faut rendre.

4.2.1 Mon fichier TAR est trop gros pour Teide !

Si vous avez ajouté des fichiers binaires dans votre historique `.git`, ou bien que vous utilisez VS Code, ou bien un éditeur de Jet Brain, vous aurez peut-être des répertoires cachés `.git`, `.vscode` ou `.idea` qui font grossir votre TAR.

Vous pouvez éditer la liste des fichiers exclus dans le `CMakeLists.txt` avant de créer le fichier TAR. Ou bien, à postériori, vous pouvez éditer le TAR, soit en ligne de commande

avec `gunzip`, `tar --delete` et `gzip`, soit avec un gestionnaire de fichiers comme `nautilus`, `thunar` ou `emacs`, ou un gestionnaire d'archive comme `ark`, `emacs` ou `file-roller`.

4.3 Architecture du lecteur

Pour simplifier votre codage, le fichier est lu deux fois simultanément par deux threads. L'un va lire, décoder et jouer le flux audio (vorbis). L'autre va lire et décoder le flux vidéo (theora), puis il va copier chaque image à un autre thread qui est responsable de l'affichage (construction des textures) au bon moment de chaque image.

Afin de ne pas utiliser trop de mémoire à la fois, ces threads dorment (`SDL_Delay(...)`) en attendant la prochaine action à faire. Pour cela, ils utilisent une référence initiale commune de temps et l'horloge temps-réel du système.

La partie audio est presque complètement gérée par la bibliothèque SDL2. Cette bibliothèque utilise elle-même des threads pour communiquer avec la carte audio.

5 Le code à réaliser

Vous devez ajouter les différents codes de gestion des threads et de synchronisation. La grande majorité de votre code de synchronisation par moniteurs sera ajouté dans le fichier `synchro.c` et `synchro.h`, mais pas uniquement.

N'oubliez pas de déclarer vos variables dans un `.c` et mettre les `extern` exposant les variables que vous souhaitez partager entre différents fichiers dans les fichiers `.h` correspondant.

5.1 Lancement et terminaison des threads

Dans la fonction `int main(int argc, char *argv[])` du fichier `ensivideo.c`, ajouter le lancement de deux threads qui exécutent, chacun, une des fonctions `theoraStreamReader` et `vorbisStreamReader`. Chacun reçoit en argument le nom du fichier à lire (`argv[1]`).

Vous devez aussi lancer le thread gérant l'affichage en exécutant la fonction `draw2SDL`. Le lancement a lieu vers la ligne 144 du fichier `stream_common.c` dans la fonction `decodeAllHeaders`. Il prend en argument le numéro du flux vidéo (`s->serial`).

Il faudra ensuite, dans la fonction `main`, attendre la terminaison du thread vorbis.

Le décodeur audio garde 1 seconde de marge. Après cette seconde de marge (le `sleep(1)` dans le code du `main`).

Dans la version PThread, vous tuerez les deux threads videos (avec `pthread_cancel`) avant d'attendre leur terminaison. Il est plus difficile de faire la même chose avec les variantes C et C++, et sauf sur une machine très lente, ce n'est pas utile.

5.2 Protection de la hashmap stockant les données de chaque flux

Les pointeurs vers les états des différents décodeurs sont stockés dans deux structures de type `struct streamstate`. La structure, coté vidéo, est maniée par deux threads. Il faudra donc la protéger des accès concurrents. Par simplification, la même fonction servant pour le décodeur vorbis, vous pouvez protéger les deux accès.

Le code est à ajouter à la fin de la fonction `getStreamState()` du fichier `stream_common.c` et au milieu de la fonction `draw2SDL()` du fichier `ensitheora.c`.

5.3 Attente et Producteur-consommateur

Il y a deux groupes de synchronisations qui correspondent à deux moments.

5.3.1 Affichage de la fenêtre vidéo

C'est le thread décodant le flux qui connaît la taille de l'image à afficher, il doit donc transmettre cette taille (en affectant les variables globales `windowSX` et `windowSY`) et attendre la création de la fenêtre avant de poursuivre.

Vous coderez les synchronisations des fonctions suivantes :

coté décodeur : `void envoiTailleFenetre(th_ycbcr_buffer buffer)` et
`void attendreFenetreTexture()` ;

coté afficheur : `void attendreTailleFenetre()` et
`void signalerFenetreEtTexturePrete()` .

Afin d'accéder aux informations de taille de la fenêtre à partir d'une variable `buffer` de type `th_ycbcr_buffer`, vous utiliserez `buffer[0].width` ainsi que `buffer[0].height`.

5.3.2 Producteur-consommateur de textures

Le fait d'avoir un seul consommateur et un seul producteur rend le code plus simple qu'un producteur-consommateur complet. Vous devez uniquement maintenir le nombre de textures déposées et pas encore affichées. La gestion du tampon de textures (ce qui est lu et écrit) avec les indices `tex_iwri`, `tex_iaff` est déjà codée par le squelette.

Le nombre maximal de textures stockables est `NBTEX`.

Vous coderez les synchronisations des fonctions suivantes :

- `void debutConsommerTexture()` ,
- `void finConsommerTexture()` ,
- `void debutDeposerTexture()` ,
- `void finDeposerTexture()` .